Master 1 - Distributed Artificial Intelligence

Data Science

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Project Report

Information Retrieval Engine

Phase 1

Aaron AIDOUDI

Academic Year 2025-2026

1. Introduction **3**

2. Dataset Description **3**

2.1 Overview and Statistics **3**

2.2 Content field creation **4**

3. Phase 1: Retrieval Basics **5**

3.1 TF-IDF Retrieval **5**

i. How It Works 5

ii. Implementation 5

iii. Results and Analysis 5

3.2 BM25+ Retrieval **6**

i. How It Works 6

ii. Implementation 7

iii. Results and Analysis 7

3.3 Embeddings Retrieval **8**

i. Purpose of High-Dimensional Embeddings 8

ii. Model Selection 8

iii. Embedding Generation and Structure 8

iv. 2D Visualization 8

v. Retrieval Results 9

3.4 Evaluation Suite **10**

i. Metrics Definition 10

ii. Comparative Results 10

iii. Discussion 11

iv. Effect of k on Retrieval Strategy 12

4. Conclusion **13**

1. Introduction

Information retrieval is one of the most fundamental challenges in data science and machine learning. At its core, the problem is deceptively simple: given a collection of documents and a user query, return the most relevant results as quickly and accurately as possible. Yet this challenge underlies the systems we interact with daily, search engines, question answering platforms, recommendation engines, and technical documentation tools all depend on the ability to efficiently match information needs with relevant content. This project explores the **design** and **evaluation** of a complete **information retrieval pipeline** applied to a multi-domain corpus of technical question-and-answer documents.

The project is organized into two phases. The first phase establishes the **retrieval foundation** by implementing and comparing three distinct approaches: TF-IDF, a classical vector space model based on term statistics; BM25+, a probabilistic ranking function with improved length normalization; and dense text embeddings generated by pre-trained transformer models, which capture semantic meaning beyond surface-level word matching. Each method is **evaluated** using standard retrieval metrics such as recall, precision, and mean reciprocal rank, to provide a quantitative basis for comparison.

The second phase introduces a document and query **classifier** trained to predict the domain category of any input text. This classifier is then integrated into the retrieval pipeline as a **reranking** mechanism, filtering and rescoring candidates based on whether their predicted category aligns with the query's domain. This approach reflects a broader principle in modern IR systems: combining multiple signals like lexical, semantic, and categorical tends to outperform any single method in isolation.

Throughout both phases, design decisions are motivated by empirical results on the training query set, and the final system is evaluated on a held-out test set through a Kaggle competition, providing an objective, external measure of retrieval quality.

2. Dataset Description

2.1 Overview and Statistics

The dataset consists of three components: a **document corpus** of 216,041 entries, a set of 327 **training queries** with **ground truth** annotations, and a separate **test query** set used exclusively for the Kaggle competition. Each document carries five fields: a unique identifier, a title, a body text, a list of tags, and a category label indicating its origin. The five categories are **tex** (31.6%), **unix** (21.9%), **gaming** (21.0%), **programmers** (14.9%), and **android** (10.6%), representing a moderately imbalanced distribution across technical domains. Queries share the same structure but consistently have an empty title field, meaning their content is carried almost entirely by the text and tags.

Document lengths vary considerably. The mean length is 973 characters with a median of 630, and a standard deviation of 1,362, indicating a right-skewed distribution with a long tail of very detailed answers reaching up to 32,650 characters. At the other extreme, some documents are as short as 54 characters, typically consisting of a title and a single tag with no body text. This heterogeneity is an important characteristic: retrieval methods must handle both short keyword-dense entries and long narrative explanations without systematically favoring one over the other.

Queries are substantially **shorter**, averaging 48.5 characters or approximately 8 words. This asymmetry between query length and document length is typical of information retrieval settings and has practical implications for all three retrieval methods we implement: **TF-IDF** and **BM25+** are sensitive to this **mismatch** through their term frequency statistics, while **embedding** models must project both short queries and long documents into a **shared semantic space**.

The **ground truth** file maps each training query to a list of relevant document identifiers, along with a relevance score and the query's category. The number of **relevant documents per query** ranges from 4 to 262, with a mean of approximately 9 and a median of 5. Most queries have between 4 and 10 relevant documents, but a small number of outliers with 50 or more relevant documents will disproportionately influence average recall scores, since finding relevant documents is inherently easier when more exist. This distributional property must be kept in mind when interpreting evaluation results.

The **category distribution** among **queries** differs from that of documents. Tex queries represent 39.8% of the training set, well above the 31.6% share of tex documents in the corpus. Conversely, gaming queries account for only 12.5% despite gaming documents representing 21.0% of the corpus. This mismatch means that evaluation metrics computed over all queries will reflect the **tex domain more heavily than others**, potentially masking weaker performance on **underrepresented categories**.

2.2 Content field creation

Not all documents contain all three text fields. Some lack a title, others have no body text, and a small number have empty tag lists. To ensure a unified representation for all retrieval methods, we **merge** the available fields into a single content string for both documents and queries.

The merging strategy follows a fixed ordering: the **title** is placed first when present, followed by the **body text**, and finally the **tags** joined as space-separated keywords. This ordering reflects an implicit priority, as titles tend to concentrate the most discriminative topical information in the fewest words. Tags are appended last rather than omitted, since they often capture domain-specific terminology that may not appear elsewhere in the document, such as specific software names, command-line tools, or LaTeX package identifiers. For documents or queries with missing fields, the merge function silently skips empty entries, avoiding the introduction of null values or redundant whitespace.

We deliberately avoided "**aggressive**" **preprocessing** steps such as stemming, stopword removal, or case normalization at this stage. Stemming can conflate terms that are semantically distinct in technical domains (for example, *"logging"* and *"log"* mean different things in a Unix context), and stopword removal may discard words that carry meaning in short queries. More importantly, embedding-based methods benefit from preserving the full surface form of text, as pre-trained transformer models have learned to handle function words and morphological variation internally. For the lexical methods, any preprocessing that might improve performance can be applied independently within each retrieval module rather than at the dataset level.

3. Phase 1: Retrieval Basics

3.1 TF-IDF Retrieval

i. How It Works

TF-IDF, short for **Term Frequency-Inverse Document Frequency**, is a classical weighting scheme that represents both documents and queries as **sparse vectors** in a **high-dimensional term space**. Each **dimension** corresponds to a unique **term** in the vocabulary, and the weight assigned to that term in a given document reflects two competing signals.

Term Frequency (TF) measures how often a term appears within a specific document, under the assumption that frequent terms are more topically relevant to that document. **Inverse Document Frequency** (IDF) counterbalances this by penalizing terms that appear in many documents across the corpus. A term like *"the"* or *"is"* carries almost no discriminative value because it appears everywhere, while a term like *"tikz"* or *"grep"* appearing in only a fraction of documents is far more informative. The final TF-IDF weight is the product of both components, producing a representation that emphasizes terms that are both locally frequent and globally rare.

Retrieval is then performed by computing the **cosine similarity** between the **query vector** and **every document vector**. Cosine similarity measures the angle between two vectors rather than their magnitude, which ensures that shorter documents are not unfairly penalized for having lower raw term counts.

ii. Implementation

We vectorized the corpus using *scikit-learn*'s *TfidfVectorizer* with the following configuration: english **stop words** were removed to **reduce noise** from common function words, document frequency thresholds were applied to filter out **terms** appearing in more than **85%** of documents (too common to be discriminative) or in **fewer** than **2** documents (too rare to generalize), and bigrams were included alongside unigrams to capture short multi-word expressions such as *"sd* card" or *"file system"*. The vocabulary was capped at 50,000 features. The same fitted vectorizer was used to transform both documents and queries, ensuring that query vectors live in the same term space as document vectors. Similarities were computed query by query to avoid materializing the full similarity matrix.

We tested **different settings**, for example lowering the document frequency threshold to a maximum of 0.95, but none of these adjustments significantly improved search quality. This initial negative result suggests that the maximum performance of TF-IDF on this dataset is limited by the method's architecture rather than by the parameter settings.

iii. Results and Analysis

The table opposite reports the average recall, precision, and MRR across all 327 training queries for three values of ***k***.

k	**Recall**	**Precision**	**MRR**
10	0.0791	0.0486	0.1367
50	0.1662	0.0212	0.1462
100	0.2169	0.0142	0.1471

The results reveal a clear **trade-off** between **recall** and **precision** as ***k* increases**. Retrieving more documents improves recall, since a wider net captures more of the relevant set, but precision falls proportionally because the additional retrieved documents are mostly irrelevant. **MRR**, by contrast, remains almost **flat** across *k* values, which indicates that TF-IDF finds its first relevant document at a relatively consistent rank regardless of how many documents are retrieved overall.

At ***k*=100**, **recall** reaches only **0.217**, meaning that on average, fewer than one in four relevant documents are found within the top 100 results. Examining individual queries helps explain this. For a query asking *"how to reformat a damaged SD card"*, TF-IDF returns five documents explicitly mentioning *"SD card"* and *"damaged"* in their titles, yet only one of the four ground truth documents appears in the top 100. The three missed documents describe the same problem using different vocabulary: *"corrupted memory card"*, *"USB storage error"*, and *"filesystem recovery"*. Since TF-IDF requires exact lexical overlap, these synonymous phrasings receive near-zero similarity scores despite being directly relevant.

A similar pattern appears in queries containing highly specific technical identifiers. Queries using LaTeX package names or Unix command flags tend to either match very precisely (when the exact string appears in relevant documents) or fail entirely (when the relevant documents paraphrase the concept). This behavior produces **high variance in per-query recall** and contributes to a **low overall average**.

The **category coherence** of retrieved results is, however, a strength of TF-IDF. Domain-specific vocabulary is sufficiently distinctive that nearly all top-ranked results belong to the correct category, even without any explicit category signal.

Overall, TF-IDF establishes a functional but limited baseline. Its recall ceiling of approximately 0.22 at *k*=100 reflects the **fundamental constraint** of **bag-of-words matching**: without any mechanism to bridge vocabulary differences, a large fraction of relevant documents will remain systematically out of reach regardless of how many results are retrieved.

3.2 BM25+ Retrieval

i. How It Works

BM25+ is a **probabilistic ranking function** that builds on the same intuitions as TF-IDF but addresses two of its known weaknesses through a more principled mathematical formulation. Like TF-IDF, it weights terms based on their frequency within a document and their rarity across the corpus, but it introduces two key refinements.

The first is term frequency saturation. In TF-IDF, term frequency contributes linearly to the document score, meaning a document that mentions a query term fifty times scores proportionally higher than one that mentions it five times. BM25+ replaces this linear relationship with a **saturation function** controlled by a parameter ***k1***, which causes term frequency contributions to plateau beyond a certain point. This prevents documents that simply repeat query terms from dominating the ranking over documents that discuss the topic more substantively.

The second refinement is **explicit document length normalization**, controlled by a parameter ***b***. Longer documents naturally contain more terms and therefore accumulate higher raw term frequencies. BM25+ adjusts each document's term frequency based on its length relative to the average document length in the corpus, ensuring that a term appearing once in a short document and once in a very long document receives appropriately different weights.

The + variant of BM25 adds a small **floor** to **term contributions**, guaranteeing that every matching term makes a positive contribution to the score regardless of document length, which avoids an edge case in the original BM25 formulation where very long documents could receive negative scores for some terms.

ii. Implementation

BM25+ requires **tokenized** input rather than vectorized representations, so documents and queries are first **lowercased** and **stripped of punctuation** before being split on whitespace. As with TF-IDF, we applied **stop word removal** at the tokenization stage alongside punctuation normalization, reducing noise from common function words while preserving domain-specific terms. The index was built using the *rank-bm25* library with standard parameter values $k1$=1.5 and b=0.75, which have been shown empirically to perform well across a wide range of retrieval tasks. Retrieval was performed by computing BM25+ scores for all documents against each query and returning the top-k highest-scoring entries.

iii. Results and Analysis

The following table reports the average recall, precision, and MRR across all 327 training queries.

k	Recall	Precision	MRR
10	0.1178	0.0716	0.1778
50	0.2010	0.0264	0.1851
100	0.2434	0.0158	0.1861

BM25+ consistently **outperforms** TF-IDF across all metrics and all values of k. At k=100, recall improves from 0.217 to 0.243, MRR from 0.147 to 0.186, and precision from 0.014 to 0.016. The improvement in MRR is particularly meaningful: a relative gain of roughly 27% suggests that BM25+ not only retrieves slightly more relevant documents overall, but also places the first relevant hit earlier in the ranking.

Despite these improvements, the two methods share the same fundamental **limitation**. Examining queries where BM25+ fails reveals the **same vocabulary mismatch** problem that plagued TF-IDF. For a query asking how to automatically reject certain types of calls, BM25+ retrieves documents about auto-reply SMS and call blocking features, which are topically adjacent, yet the four ground truth documents do not appear in the top 100. The relevant documents apparently describe the same functionality using sufficiently different phrasing that no lexical signal survives.

A notable behavioral difference is that BM25+'s **length normalization** sometimes penalizes short but highly relevant documents. In TF-IDF, a short document with exactly the right terms can score very highly due to the concentrated term density. In BM25+, the same document may score lower because its brevity causes term frequencies to be up-weighted against corpus averages, occasionally pushing it below longer, more verbose documents that discuss the topic at greater length. This effect is most visible in categories like android and gaming, where many relevant documents are short community answers rather than detailed technical explanations.

At k=100, BM25+ recall reaches 0.243, representing an improvement over TF-IDF but still leaving approximately three quarters of relevant documents unfound. Both lexical methods appear to share a **recall ceiling** in the low-to-mid twenties at this retrieval depth, suggesting that further parameter tuning will not break through this barrier. The **structural limitation** of term matching without semantic understanding motivates the move to embedding-based retrieval in the next section.

3.3 Embeddings Retrieval

i. Purpose of High-Dimensional Embeddings

Dense text embeddings represent a fundamentally different approach to information retrieval. Rather than matching documents and queries based on shared vocabulary, embedding models convert text into dense **numerical vectors** in a **high-dimensional space** where geometric proximity reflects semantic similarity. Two texts that express the same idea using entirely different words will have vectors that are close together, while two texts that share many words but discuss different topics may end up far apart.

This property emerges from the training process of transformer-based embedding models. Models like *SentenceTransformers* are trained on large **collections** of **semantically related text pairs**, learning to map similar meanings to nearby regions of the vector space. As a result, the embeddings capture relationships that lexical methods cannot: synonyms, paraphrases, domain-specific analogies, and contextual nuances. In an information retrieval context, this means that a query asking *"how to fix a corrupted memory card"* can successfully retrieve a document titled *"SD card is damaged"* even though no word is shared between them. This is precisely the vocabulary mismatch problem that caused both TF-IDF and BM25+ to miss large portions of the relevant document set.

Retrieval then proceeds in the same way as before: for each query vector, we compute cosine similarity against all document vectors and return the top-k highest-scoring documents. The difference is that this similarity now measures semantic closeness rather than lexical overlap.

ii. Model Selection

We evaluated several models from the *SentenceTransformers* library, as well as models from other families available on the *HuggingFace* hub. Our baseline was ***all-MiniLM-L6-v2***, a lightweight **22-million parameter** model producing **384-dimensional embeddings** with a maximum sequence length of **256 tokens**. Its compact size made it an efficient starting point, encoding the full 216,041-document corpus on a Tesla T4 GPU in approximately **10 minutes**.

We then tested a range of larger and more specialized models, including retrieval-focused architectures and models with higher-dimensional output spaces. Several candidates were either too slow for practical use within our time budget or produced worse results despite their larger size, likely due to architecture choices or training data that did not align well with our multi-domain technical corpus. After experimentation, we retained ***all-MiniLM-L12-v2*** as our primary model. This **12-layer** variant of the same MiniLM architecture shares the same 384-dimensional output and 256-token limit as the L6 baseline but adds six additional transformer layers, encoding approximately 33 million parameters. The increased depth allows the model to capture more complex semantic relationships, at the cost of roughly **15 minutes** of encoding time rather than 10.

iii. Embedding Generation and Structure

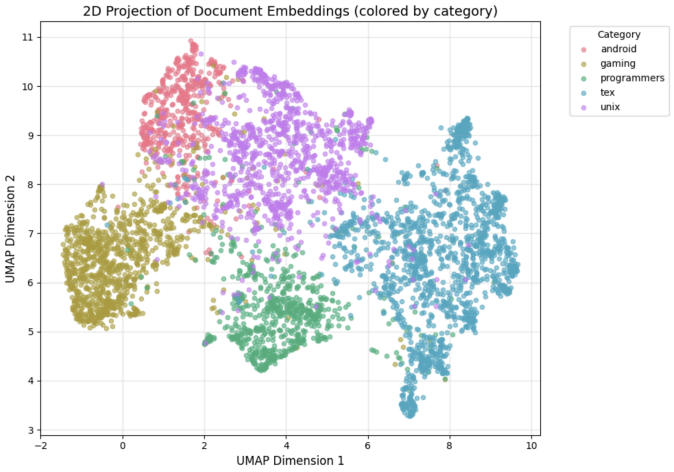
Embeddings were generated for all documents and queries using a **batch size** of **128**. Individual embedding vectors are dense float32 arrays with values distributed tightly around zero (mean 0.0009, standard deviation 0.051).

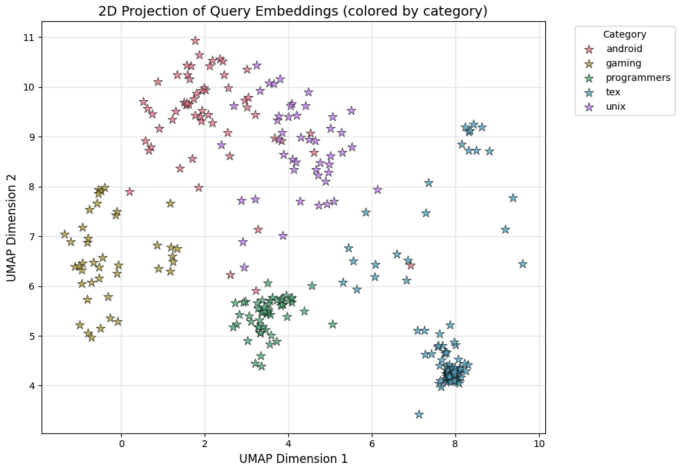
iv. 2D Visualization

To verify that the embeddings capture meaningful structure, we projected a stratified **sample** of 5,000 documents into two dimensions using **UMAP** with **cosine distance**. The following resulting

visualization reveals clear clustering by category: tex documents form a compact and well-separated region, reflecting the distinctive vocabulary of LaTeX commands and typesetting terminology. Android and Unix documents occupy adjacent but partially overlapping regions, which is expected given that the two domains share considerable vocabulary. Programmers and gaming documents form their own clusters with moderate separation from the others.

This category-level clustering has a direct implication for retrieval. When query and document embeddings are projected into the same space, queries naturally land close to the cluster corresponding to their domain, which means that even without any explicit category signal, the embedding similarity scores implicitly perform a soft domain filtering. However, the partial overlap between adjacent categories also explains occasional cross-category retrievals, such as a unix document appearing in the top results for an android query when the underlying problem (file system operations, for instance) transcends platform boundaries.





v. Retrieval Results

For each query embedding, we computed cosine similarity against all document embeddings and retrieved the top-100 results. The opposite table reports the ***all-MiniLM-L6-v2*** performance on the training query set.

k	**Recall**	**Precision**	**MRR**
10	0.1959	0.1141	0.2745
50	0.3533	0.0461	0.2839
100	0.4184	0.0284	0.2846

The **improvement** over both lexical methods is substantial. At k=100, recall reaches 0.418, nearly double TF-IDF's 0.217 and well above BM25+'s 0.243. MRR improves to 0.285, a relative gain of 93% over TF-IDF and 53% over BM25+. This improvement is not marginal, it reflects a qualitative change in what the system can find, not just a refinement of ranking within the same retrieved set.

Examining individual queries illustrates this concretely. For the SD card reformatting query discussed earlier, where TF-IDF found 1 out of 4 relevant documents and BM25+ found 2, the embedding model retrieved all 4, with the relevant documents appearing at ranks 9, 25, 26, and 32. The model successfully bridged terms like *"damaged"*, *"corrupted"*, and *"fix"* by recognizing their semantic proximity in the embedding space. For the call rejection query, embeddings found 3 out of 4 relevant documents with the first hit at rank 5, compared to TF-IDF's single hit at rank 80 and BM25+'s complete failure.

MRR stability across k values is also notable: the score barely changes between k=10 and 100, suggesting that when the embedding model finds a relevant document, it tends to rank it in the top 10. Increasing retrieval depth primarily surfaces additional relevant documents at lower ranks rather than improving the position of the first hit, which was already well-placed.

That said, embeddings are **not uniformly superior**. On queries with very specific technical identifiers, such as LaTeX package names or precise Unix command flags, lexical methods occasionally outperform embeddings. When a query contains a rare but exact string like a specific package identifier, TF-IDF's high IDF weight for that term produces a very precise match that embeddings may dilute by also retrieving documents about conceptually related but not identical topics. Additionally, queries whose content field contains many appended tags can introduce semantic noise that pulls the query embedding toward a generic domain representation rather than a specific information need.

Switching from all-MiniLM-L6-v2 to **all-MiniLM-L12-v2** as our final model for the Kaggle submission produced a meaningful improvement, with recall rising to 0.4868 and MRR to approximately 0.35 on the training set.

3.4 Evaluation Suite

i. Metrics Definition

We evaluate all retrieval models using three standard information retrieval metrics, each measuring a distinct aspect of system quality. All three take values in where higher is better.

Recall measures coverage: what fraction of the relevant documents were actually found within the top-k results. It answers the question of whether the system retrieves everything that matters, regardless of where in the ranking those documents appear.

Precision measures accuracy: what fraction of the retrieved documents are actually relevant. It answers the question of whether the results returned to the user are useful, penalizing systems that retrieve many irrelevant documents to find the relevant ones.

Mean Reciprocal Rank (MRR) measures ranking quality: how early the first relevant document appears in the ranked list. For each query, the reciprocal rank is 1 divided by the position of the first relevant hit. The MRR is the average of these reciprocal ranks across all queries. A system that consistently places a relevant document at rank 1 achieves an MRR of 1.0, while one that only finds a relevant document at rank 10 on average achieves 0.1

ii. Comparative Results

The three following tables report the performance of all three retrieval models across k. The **all-MiniLM-L12-v2** results are used, as it was selected as our final model following experimentation after comparing both variants on the training set.

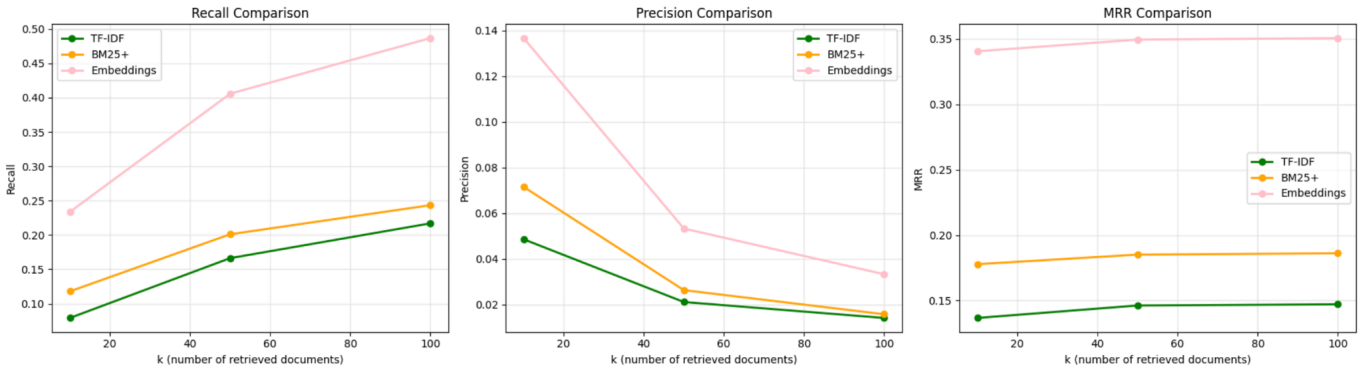
k	TF-IDF	BM25+	Embeddings (L12)
10	0.0791	0.1178	0.2336
50	0.1662	0.2010	0.4057
100	0.2169	0.2434	0.4868

Opposite table : **Recall** across retrieval methods.

k	TF-IDF	BM25+	Embeddings (L12)
10	0.0486	0.0716	0.1367
50	0.0212	0.0264	0.0533
100	0.0142	0.0158	0.0333

k	TF-IDF	BM25+	Embeddings (L12)
10	0.1367	0.1778	0.3406
50	0.1462	0.1851	0.3495
100	0.1471	0.1861	0.3507

Precision across retrieval methods / **MRR** across retrieval methods



iii. Discussion

The results confirm a clear performance hierarchy across all metrics and all values of *k*: embeddings outperform BM25+, which outperforms TF-IDF, without exception. The magnitude of the differences, however, varies considerably across metrics and methods.

The **gap** between **lexical methods** and **embeddings** is large and **consistent**. At *k*=100, embedding recall (0.487) is more than double TF-IDF recall (0.217) and roughly double BM25+ recall (0.243). MRR shows a similar pattern: the embedding model achieves 0.351 against BM25+'s 0.186 and TF-IDF's 0.147. These are not marginal improvements attributable to parameter tuning but reflect a fundamental difference in what each class of method can retrieve. Lexical methods are bounded by the vocabulary overlap between query and document, while embeddings operate in a semantic space that transcends surface-level word matching.

The **gap** between **TF-IDF** and **BM25+**, by contrast, is real but **modest**. BM25+'s recall is consistently 10 to 12 percentage points higher than TF-IDF's, and its MRR gains of roughly 25 to 30% suggest meaningfully better ranking quality. These gains are attributable to BM25+'s length normalization and saturation function, which produce more balanced scores across documents of varying lengths. However, both methods share the same recall ceiling in the low-to-mid twenties at *k*=100, confirming that their shared architectural limitation (lexical matching without semantic understanding) is the binding constraint rather than differences in term weighting formulas.

For all three methods, **precision** falls sharply as *k* increases: this is mathematically expected, since the number of truly relevant documents per query averages around 9, meaning the theoretical maximum Precision@100 is approximately 9%. The absolute precision values are therefore low by design, and should not be interpreted as evidence of poor retrieval quality. What matters is the relative ordering across methods, where embeddings maintain a consistently higher fraction of relevant documents among their retrieved results at every k.

MRR stability is another informative pattern. For all three methods, MRR changes very little between *k*=10 and 100. This indicates that the first relevant document in the ranking is found at roughly the same position regardless of how many documents are retrieved overall. Extending retrieval depth improves recall by surfacing additional relevant documents at lower ranks, but it

does not move the first hit higher in the ranking. The embedding model's MRR of 0.35 translates roughly to placing the first relevant document at rank 3 on average, compared to rank 5 to 6 for BM25+ and rank 7 for TF-IDF.

We also explored **several strategies** to push **embedding performance** further. We tested a number of alternative models, including retrieval-specialized architectures and models producing higher-dimensional representations. For example: ***all-mpnet-base-v2, multi-qa-MiniLM-L6-cos-v1, BAAI/bge-m3, BAAI/bge-small-en-v1.5***. None consistently outperformed ***all-MiniLM-L12-v2*** on our training set within acceptable time constraints, and several produced noticeably worse results despite their larger parameter counts, likely because their training distributions did not align well with our multi-domain technical corpus. We also experimented with **combining multiple models** through reciprocal rank fusion, and with re-encoding the top-100 candidates using a second-pass embedding computed over the original query and the retrieved document text directly. Neither approach yielded consistent improvements over the single-model L12 baseline, and both introduced significant additional complexity and computational cost. We therefore retained *all-MiniLM-L12-v2* as the sole retrieval backbone for the Phase 1 Kaggle submission.

iv. Effect of *k* on Retrieval Strategy

The choice of ***k*** involves a **trade-off** that depends on how retrieval output will be used. A small ***k*** favors **precision** and is appropriate when downstream processing is expensive or when the end user only inspects a short result list. A large ***k*** favors **recall** and is preferable when the goal is to feed a reranking stage with as many candidates as possible.

In the context of the Kaggle scoring formula, where **recall**, **precision**, and **MRR** each contribute equally at **25%**, increasing *k* produces a net positive effect on the final score as long as the recall gain outweighs the precision loss. Since MRR is nearly flat across *k* values for the embedding model (0.3505 at *k*=100 versus 0.3510 at *k*=250), the score dynamics simplify to a competition between recall growth and precision decay. Recall grows substantially faster: moving from *k*=100 to *k*=250 adds 11.6 percentage points of recall while only losing 2 percentage points of precision. This asymmetry holds because the average query has approximately 9 relevant documents out of 216,041 total, meaning each additional retrieved document has a very small marginal effect on precision but a measurable chance of being relevant and boosting recall.

From this analysis, we systematically explored **higher values of *k*** and observed increasingly larger results for recall. At the same time, we posted these results on Kaggle and the score increased monotonically from **0.214 at *k*=100 to 0.253 at *k*=700, even to 0.312 at *k*=10,000**. Our submitted results therefore represent a practical operating point on the recall-precision trade-off curve.

That said, **we acknowledge that this reflects an optimization of the evaluation metric rather than of retrieval quality per se**. The genuine retrieval quality of our model is more meaningfully captured by two complementary views: **MRR, which remains stable across all *k* values and is entirely independent of retrieval depth inflation**, and **the metrics at *k*=100, which represent a realistic operating point for a deployed retrieval system** (with 0.48 recall, 0.033 precision and 0.35 MRR). **Returning 10,000 documents per query is not a viable design choice in any practical setting, and the metrics computed at that depth should be interpreted accordingly.**

4. Conclusion

This **first phase** of the project established a **complete information retrieval pipeline** covering the spectrum from classical lexical methods to modern semantic representations. Starting from a raw multi-domain corpus of 216,041 technical documents, we implemented three distinct retrieval approaches, developed a standardized evaluation suite, and compared their performance quantitatively.

The results tell a coherent story. **TF-IDF** and **BM25+** provide **fast**, **interpretable** baselines that perform well within their architectural limits, but share a fundamental **ceiling** imposed by the **vocabulary mismatch** problem. Dense **embeddings**, by contrast, retrieve semantically relevant documents regardless of surface-level phrasing differences, producing recall and MRR figures roughly double those of the lexical methods. Among the embedding models tested, ***all-MiniLM-L12-v2*** offered the best balance of quality and encoding speed on our hardware, and was retained as the backbone of our Phase 1 Kaggle submission. The systematic exploration of retrieval depth k further revealed that increasing the number of retrieved documents consistently improves the Kaggle score, since recall grows faster than precision decays under the competition's equal-weight scoring formula.

The **second phase** will build on this foundation in two directions. First, we will train a document and query classifier to predict domain categories. Second, we will use this classifier to perform reranking.